清華大學
Tsinghua University

组会汇报： RecreationWorld

Scalable and Verifiable Environments for Hybrid Computer-Use Agents

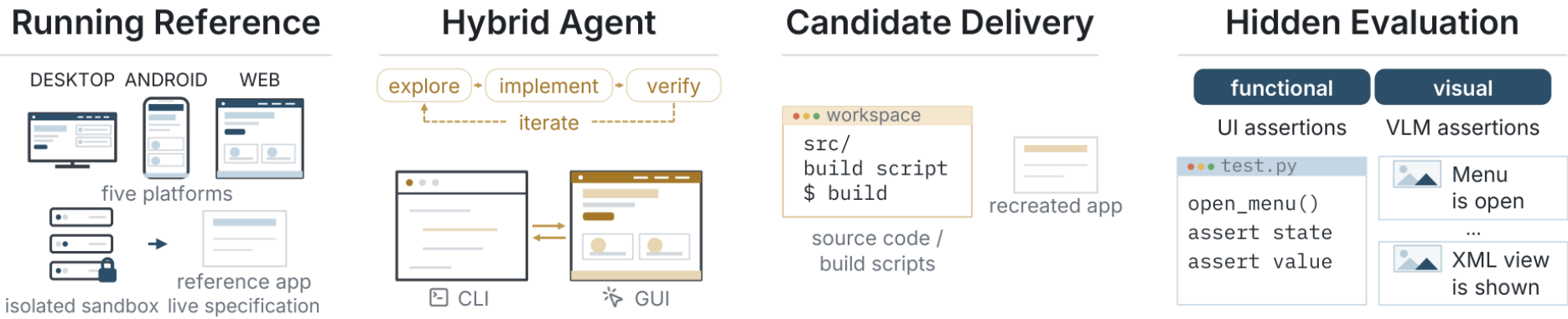
面向混合计算机操作智能体的可扩展、可验证环境

汇报人：周博文 2025213964



为什么选择“应用复刻”作为任务

□ **Recreation： 给定运行中的参考应用，探索其行为并实现一个行为一致的复刻版**



必须混合（Hybrid by necessity）

规格只存在于参考应用的界面里，交付物却是必须编译运行的代码；探索—实现—验证的顺序与频率不被规定。

天然可验证（Verifiable）

运行中的参考应用就是 oracle：可派生隐藏行为测试并回放到任意候选上，得到与实现无关的客观奖励。

可扩展（Scalable）

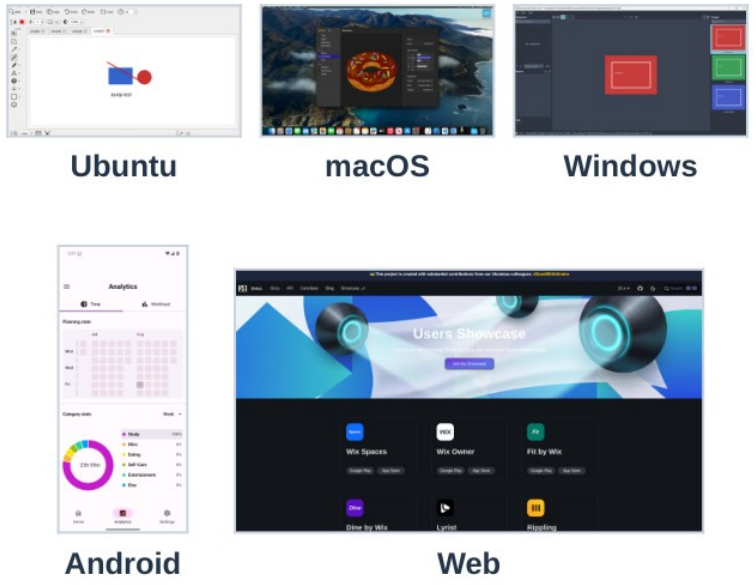
开源 GUI 应用源源不断，可转化为开放式的任务与已验证轨迹流，而非固定的人工题集。

复刻把“理解系统需要操作它、改造系统需要写代码”压缩进一个任务，同时自带客观的执行级奖励

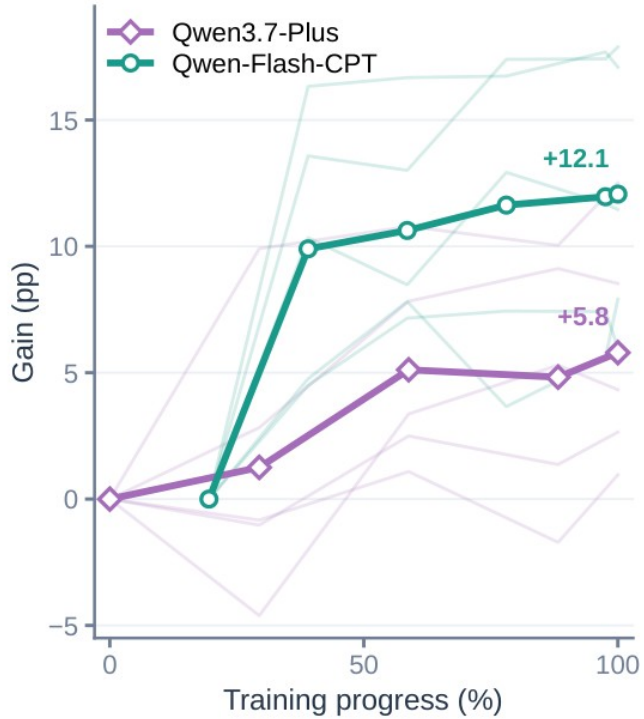
RecreationWorld 框架总览

☐ 五平台可复现环境 + 统一智能体 harness + 以参考应用为依据的隐藏测试

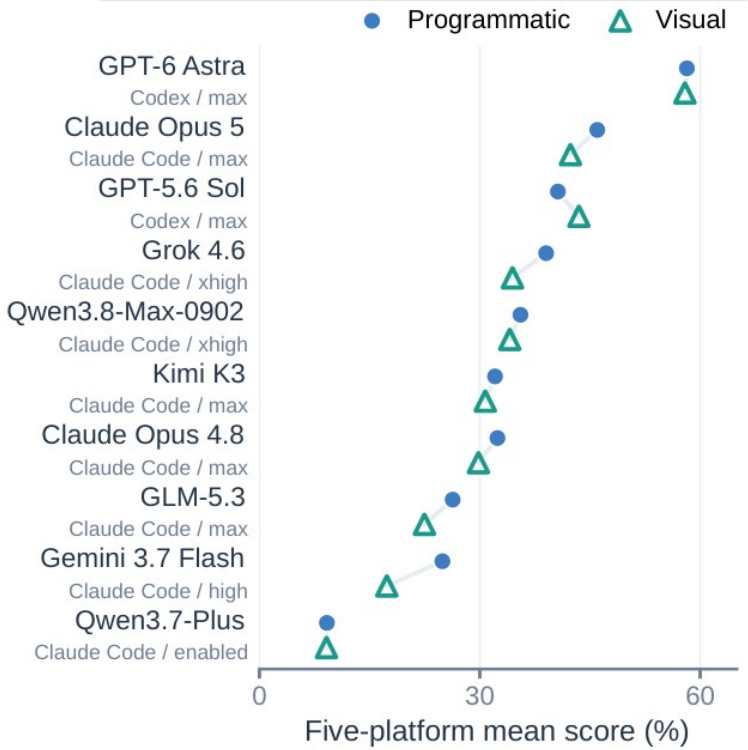
Diverse executable tasks



Scalable verified experience



RecreationBench

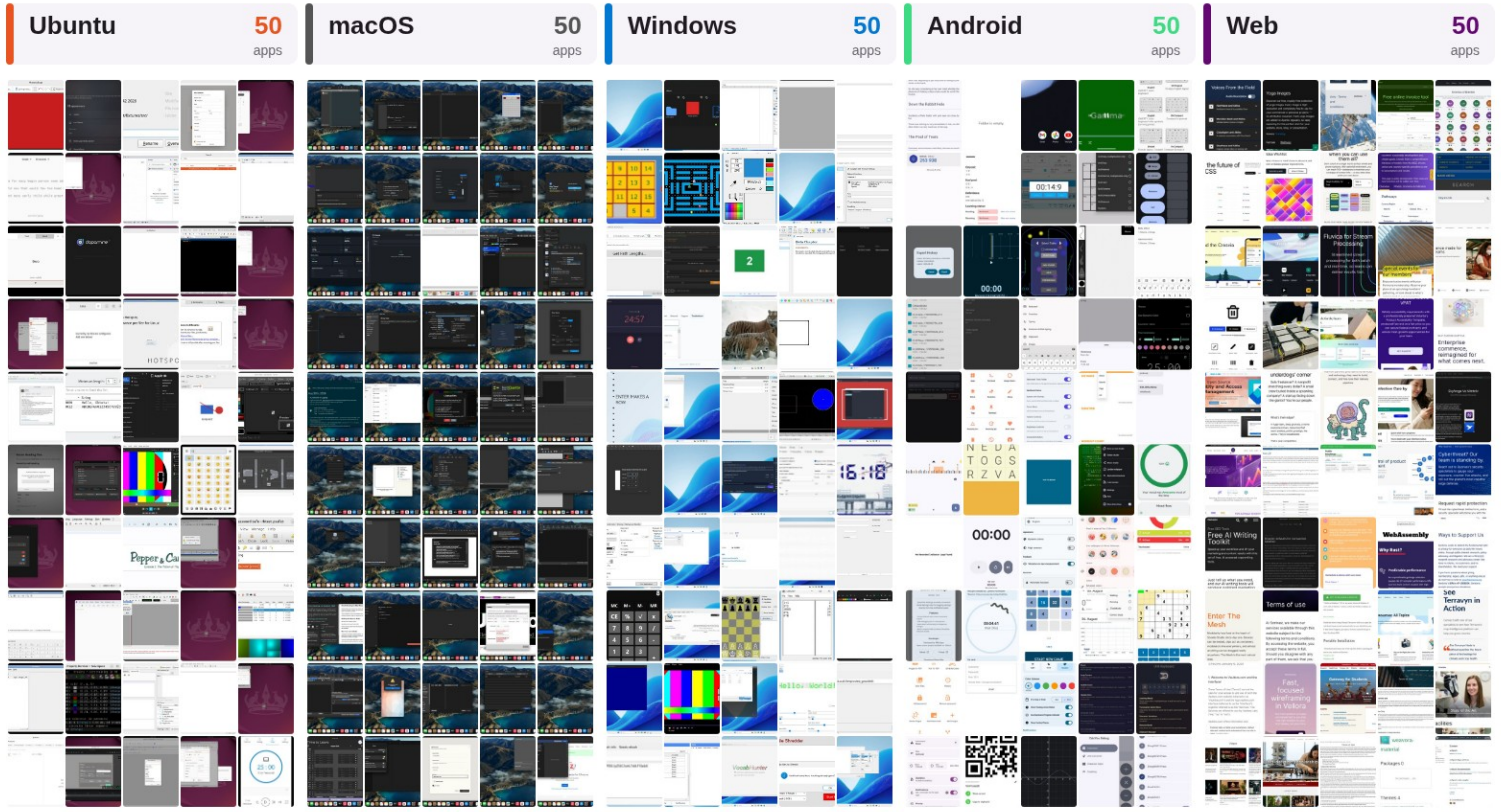


左：五平台复刻任务；中：在 5 个分布外基准上的迁移（相对各自首个已评估检查点）；右：五平台程序化与视觉得分

环境： Ubuntu · macOS · Windows · Android · Web ； 原生 GUI 控制 + 编码工具共用一个执行约定；任务隔离的 worker 在可水平扩展的虚拟机池中调度

RecreationBench： 250 个真实开源应用

□ 五个原生平台各 50 个留出应用，覆盖不同领域、 UI 框架、编程语言与复杂度

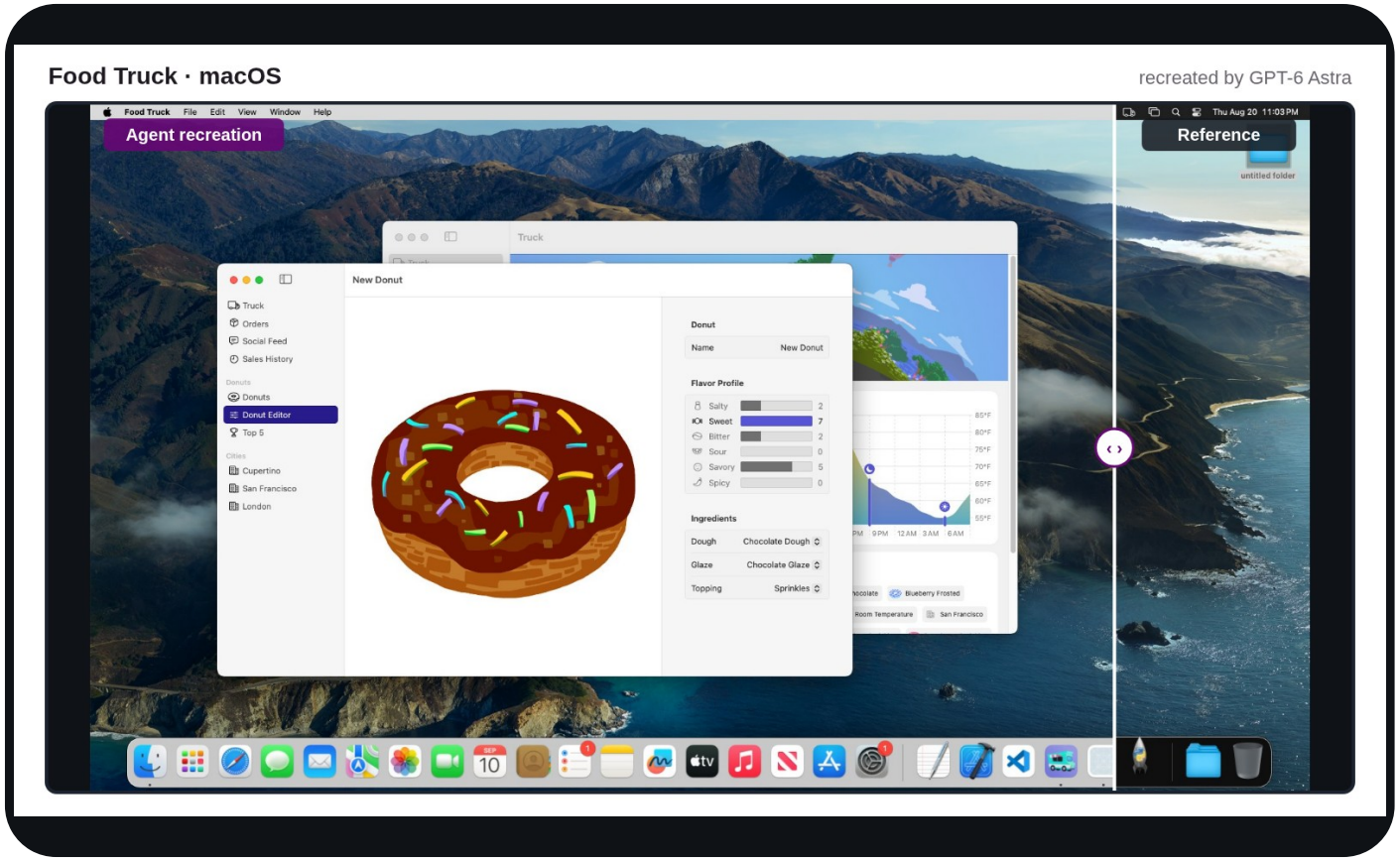


RecreationBench · 250 held-out applications · 5 native platforms

测试前先在参考应用上回放校验、再人工审核；参考实现在可能处不公开

参考应用 vs. 智能体复刻版

☐ 隐藏测试只评可观察行为，不限定候选的语言、框架与架构



参考应用 ↔ 智能体复刻版（拖动对比，数据来自 recreation-bench.cc ）

评分方式

- 由参考应用自动合成程序化断言 + 视觉断言，先在参考上回放通过、再人工审核后冻结
- 总分 = 程序化得分与 VLM 得分的均值
- 另报告程序化 ⩾ 90% / =100% 的任务比例

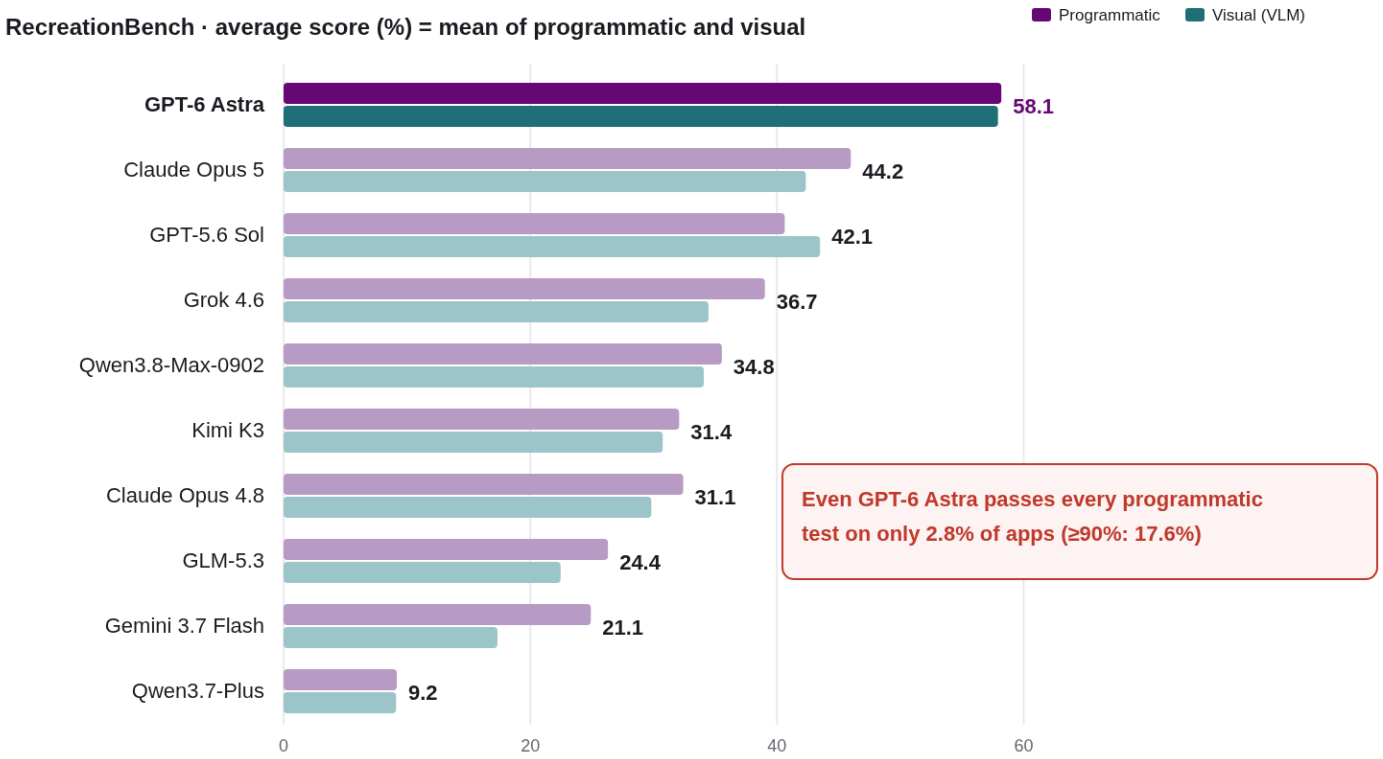
防作弊与隔离

- 各平台网络隔离，禁止运行时检索参考实现
- 统计外网访问、受保护路径、解包参考应用、提权四类违规尝试
- 不限定候选的语言、框架与架构，只评可观察行为

测试同时覆盖“看得见的结构”和“交互后的状态与计算结果”，而非仅清点界面元素

主结果： 10 个前沿模型在 RecreationBench 上的表现

☐ 平均得分 = 程序化得分与 VLM 得分的均值，五平台等权平均



58.1%

GPT-6 Astra 总分最高，领先第二名 Claude Opus 5 （44.2%） 13.9 分

仅 2.8%

即使最强模型，程序化测试 100% 全通过的任务也只有 2.8% （≥ 90% 为 17.6%）

静态结构 > 交互与计算

智能体对静态界面结构的还原明显好于交互行为和计算结果；生成的应用比参考应用小得多、更单体化

复刻任务远未饱和：前沿模型能“长得像”，但离“行为一致”还有很大差距

训练：复刻轨迹能否迁移到分布外任务？

□ Qwen3.8-Max 生成复刻轨迹 → 行为 verifier 拒绝采样 → 3.5 万条 SFT 数据

① 任务

GitHub 高质量开源 GUI 应用，覆盖五平台，与评测集去重

② 轨迹生成

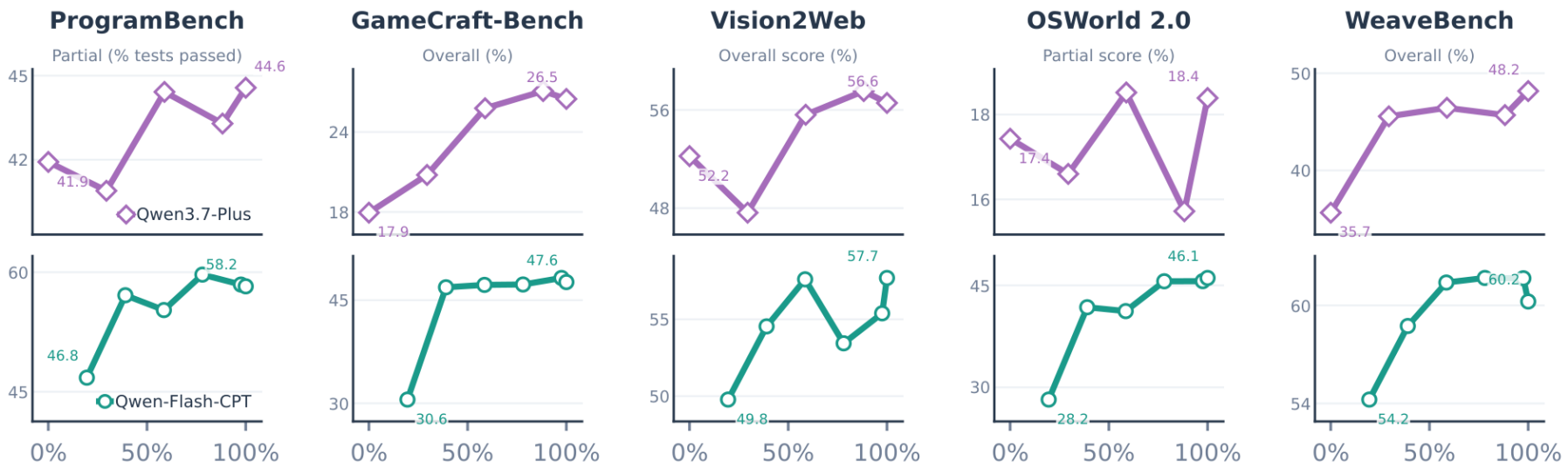
Qwen3.8-Max 在隔离 VM 池中并发执行探索—实现—构建运行—核验修正

③ 拒绝采样

任务级行为 verifier 筛选高分轨迹，每平台 7,000 条，共 3.5 万条

④ 监督微调

同一数据分别训练 Qwen3.7-Plus 与 CPT 后的 Qwen-Flash

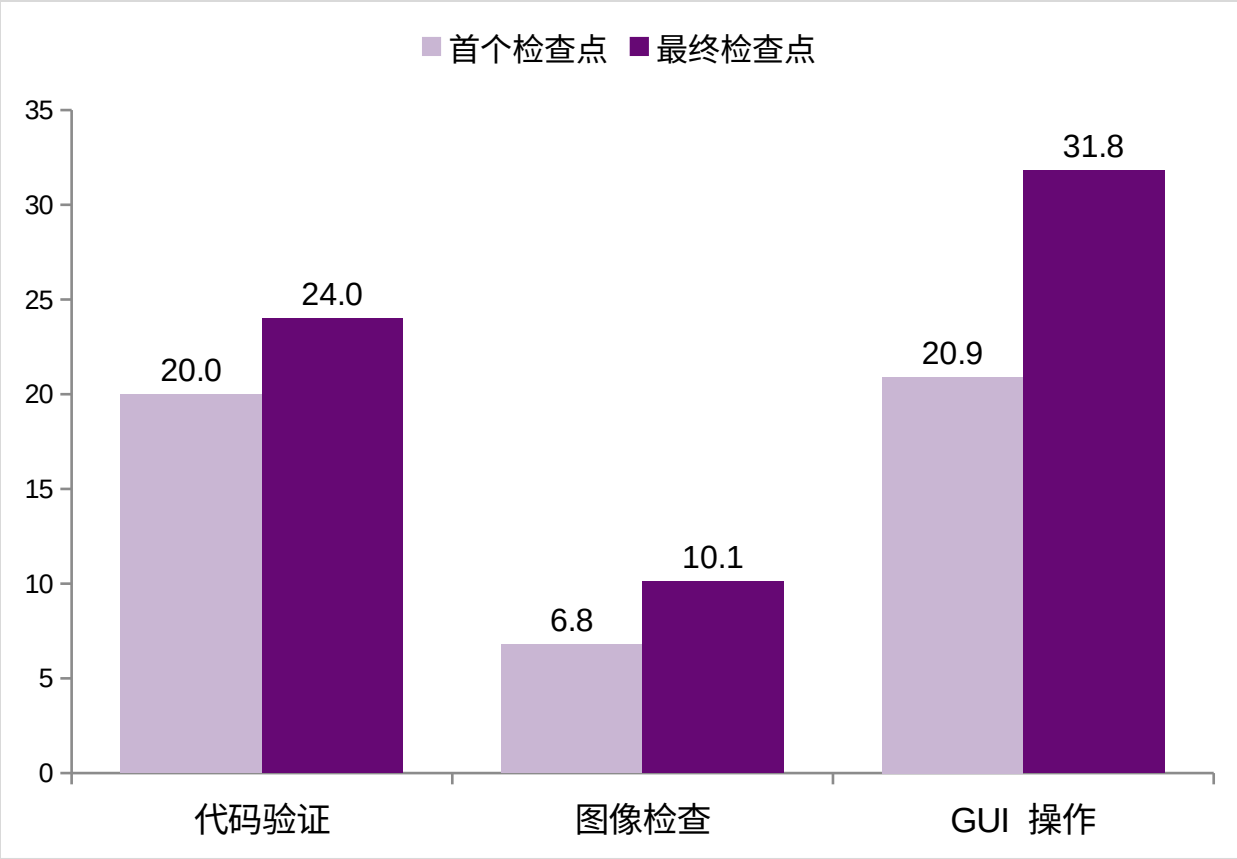


代码（ProgramBench、GameCraft）· 视觉代码（Vision2Web）· 混合计算机操作（OSWorld 2.0、WeaveBench）；横轴为归一化训练进程

两种初始化在 5 个分布外基准上均高于首个已评估检查点，最高 +17.9 分；WeaveBench 平均部分得分 35.7 → 48.2

训练后智能体更常检查自己的输出

☐ 首个检查点 vs. 最终检查点：两种初始化的平均行为占比（%）



代码验证 20.0 → 24.0

主要来自“修改代码之后”的验证（18.2 → 22.2），而非修改前

图像检查 6.8 → 10.1

既更多查看素材，也更多查看自己渲染的输出（2.9 → 4.7）

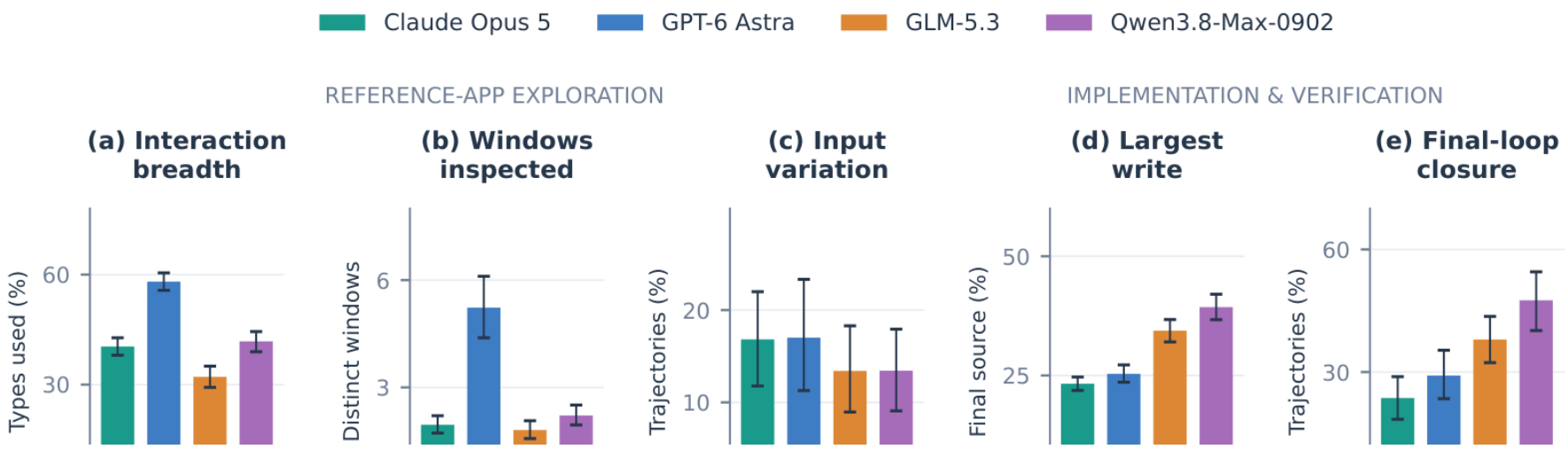
GUI 操作 20.9 → 31.8

观察（10.2 → 15.1）与交互（10.7 → 16.7）同步增加

复刻训练让智能体更主动地“运行—看结果—再修改”，为自我改进提供了基础

分析一：强模型赢在“探索广度”，所有模型都缺最后一轮验证

□ 四个模型的轨迹统计（五平台等权均值， 95% bootstrap 区间）；均为描述性对比



探索更广 ↔ 得分更高

GPT-6 Astra 覆盖 58.1% 的参考端 GUI 交互类型、观察 5.23 个不同窗口； Qwen3.8-Max 为 41.8% / 2.21

受控输入变化很少

同一输入框尝试两个以上不同值的轨迹仅 13–17%，各模型接近

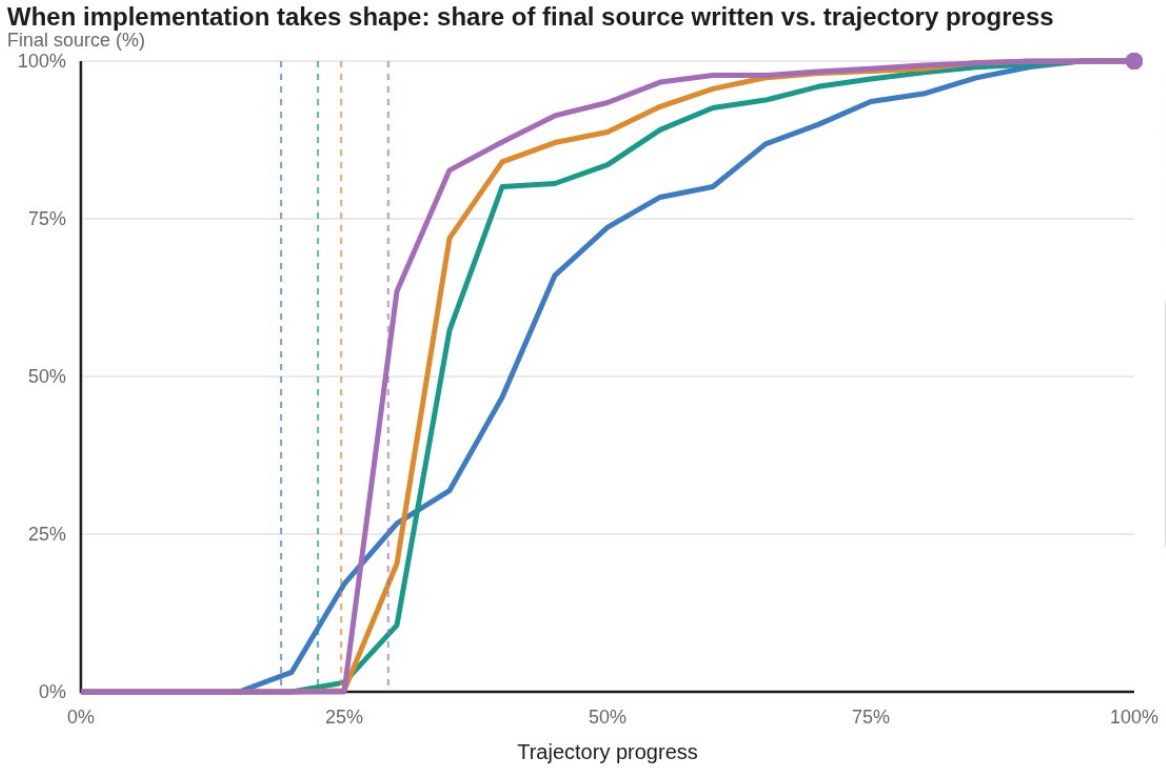
最终闭环普遍缺失

“最后改代码→重启→再观察”完成率最高 47.5% （Qwen3.8-Max）， GPT-6 Astra 仅 29.1%

差距主要在“去看了多少”，而交付前的最终核验所有模型都做得不够

分析：先探索、后实现——代码在轨迹中何时成形

☐ 各模型五平台中位曲线：已写出的源码占最终源码的比例（官网 Source growth 数据）



代码出现得晚

首次写代码的中位时刻在轨迹的 19%–29%，前段主要在探索参考应用

中段集中实现

到轨迹中点，大部分源码已经写出（GPT-6 Astra 约 74%，其余 84%–93%）

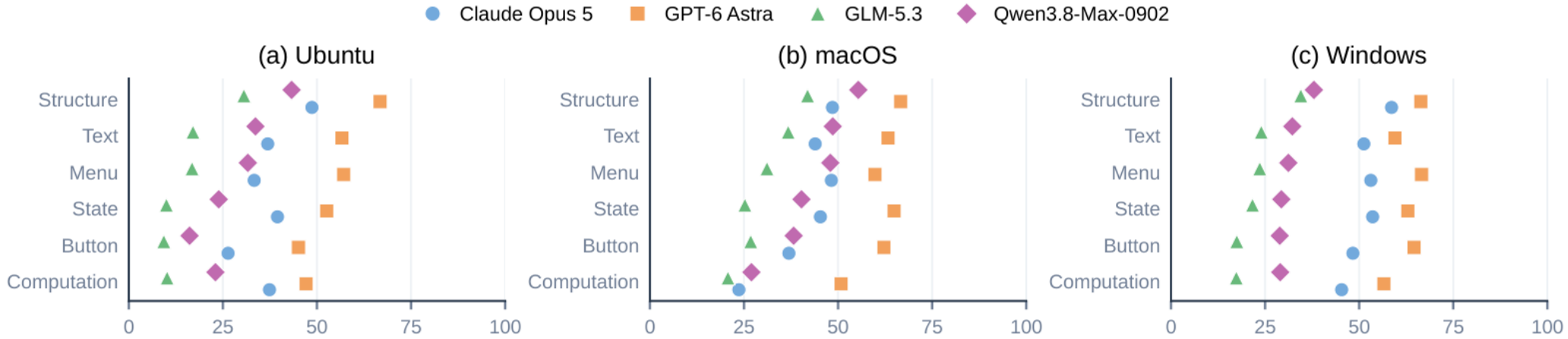
后段小步修改

后半程以较小的编辑为主； GPT-6 Astra 的曲线最平缓，边探索边增量实现

复刻任务确实形成了“探索—实现—验证”的长程循环，而不是一次性生成代码

分析二：复刻出来的应用“长得像、行为不全”

☐ 按测试类别的程序化通过率（Ubuntu / macOS / Windows 三平台示例）



结构 > 交互与计算

各平台最弱的类别总是按钮、计算或交互，比“结构”低 9.1–34.4 分； Web 上静态内容比交互高 28.2–36.7 分

例： PhotoCollage

旋转照片后撤销：布局恢复了，但照片仍是旋转状态。首屏看着对、也能启动，只有动作序列才暴露问题

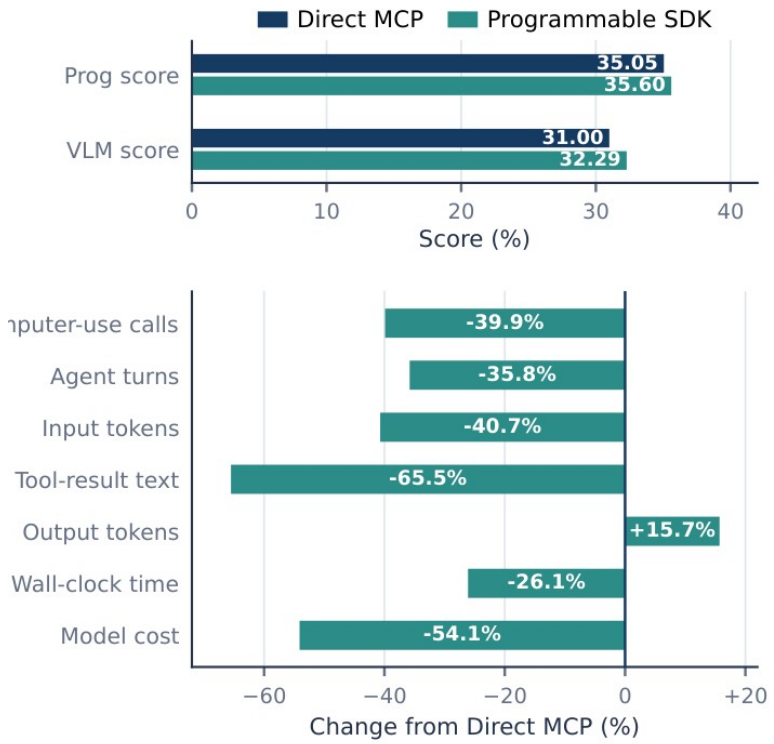
代码更小、更单体

复刻版中位文件数 4–43 ，参考应用 47–167 ；最大单文件占比 10–64% （参考 5–21% ），还常更换 UI 框架

这也是为什么要用“触发—响应”的多深度行为测试，而不是只比截图或清点控件

分析三：把 GUI 操作写成程序，能显著降本

□ Windows 50 个应用、 Claude Opus 4.8： 逐步 MCP 调用 vs. 持久 REPL + 可编程 SDK



质量基本不变

程序化 35.05 → 35.60， VLM 31.00 → 32.29

开销大幅下降

计算机操作调用 − 39.9% 、轮数 − 35.8% 、输入 token −40.7% 、工具返回文本 − 65.5%； 模型成本 − 54.1% （每题 $90.50 → $41.58 ）

原因

智能体可以用循环和条件组合多步 GUI 操作，并只把需要的观测放回上下文

混合智能体的 GUI 一侧也可以“代码化”： 这是降低长程 GUI 开销的另一条路